



نظام الكشف عن الالتهاب الرئوي من أشعة الصدر

Chest X-ray Pneumonia Detection System

شرح تقني متعمق: الـ AI pipeline كامل + الـ web application

شرح بالمصري للمشروع كله من أول الـ data لحد ما الـ model يشتغل في الموقع، بالتفاصيل والأرقام والصور.

Team 19 - School of CSAI, Zewail City of Science and Technology - Supervisor: Prof. Dr. Khaled Mostafa
June 2026 -

الورقة دي مكتوبة عشان أي حد يقرأها يقدر يفهم المشروع كله ويشرحه بثقة، حتى لو أول مرة يشوفه. فيها قسمين: القسم A بيشرح الـ AI pipeline (الـ data، الـ preprocessing، الـ models، التدريب، التقييم، والـ XAI)، والقسم B بيشرح الـ web application (الـ frontend والـ backend والـ database والـ security ورحلة الطلب من الأول للآخر). كل قسم بيتتهي ببنك أسئلة متوقعة من لجنة المناقشة بإجاباتها.

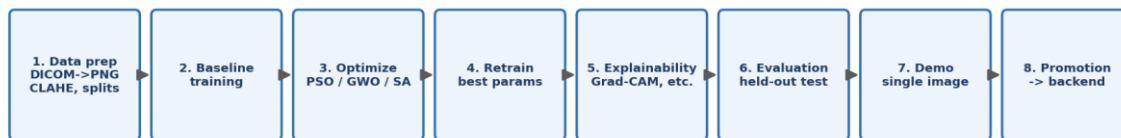
الفهرس - Table of Contents

الفهرس - Table of Contents	2
القسم A - الـ AI Pipeline	3
A1. الـ Dataset: RSNA Pneumonia Detection Challenge	3
A2. من صورة خام لـ model input (الـ Preprocessing)	3
A3. تقسيم الـ data صح + معالجة عدم توازن الـ classes (imbalance)	3
A4. مهمتين: classification و detection	4
A5. الـ Models بالتفصيل + عدد الـ parameters	4
A6. تدريب الـ classifiers (كل الإعدادات)	5
A7. تدريب الـ detectors	5
A8. الـ Nature-Inspired Optimization: PSO و GWO و SA	5
A9. مقاييس التقييم (Metrics) بشرح بسيط	6
A10. النتائج، ولية طلعت كده، وإزاي نحسنها	6
A11. الـ Explainability (XAI): نوري الدكتور 'ليه'	8
A12. القسم A - أسئلة متوقعة من اللجنة	10
القسم B - الـ Web Application Pipeline	12
B1. الـ Frontend (Angular 21)	12
B2. الـ Backend (FastAPI)	13
B3. الـ Inference Service خطوة بخطوة	13
B4. الـ Security	14
B5. الـ Database (MongoDB)	14
B6. الـ API	14
B7. رحلة طلب كاملة من الأول للآخر	15
B8. الـ Deployment والإعداد	15
B9. القسم B - أسئلة متوقعة من اللجنة	16

القسم A - ال Pipeline AI

باختصار في جملة: بناخذ صورة أشعة صدر (chest X-ray)، نعدّيها على deep-learning model مدّرب، وبيرجّلنا يا إما احتمال إن الرئة فيها التهاب رئوي (دي مهمة ال classification)، يا إما box حوالين مكان الالتهاب مع نسبة ثقة (دي مهمة ال detection). كل اللي تحت بيشرح إزاي بنوصل من صورة طبية خام للإجابة دي، وليه بتطلع كده.

AI / ML Pipeline (8 phases)



شكل A1. مراحل ال AI pipeline الثمانية (phases 1 -> 8).

A1. ال Dataset: RSNA Pneumonia Detection Challenge

استخدمنا dataset بتاع تحدي RSNA Pneumonia Detection اللي نشرته ال Radiological Society of North America فيه حوالي 26,684 صورة أشعة صدر أمامية (frontal chest X-rays)، كل واحدة اتعلّمت (labeled) من radiologists. أهم نقطة إنه بيدّينا نوعين من ال labels: label على مستوى الصورة (فيه التهاب ولا لا) بنستخدمه في ال classification، و bounding boxes (المستطيلات اللي بتحدد مكان ال opacity اللي بيشبه الالتهاب) بنستخدمها في ال detection. اخترناه لأنه كبير، متعلّم بإيد متخصصين، ومستخدّم ك benchmark مشهور (يعني أرقامنا تقدر نتقارن بغيرنا)، وببعدم المهمتين مع بعض.

ليه ده مهم؟ جودة أي model مسقوفة بجودة ال data بتاعته. وبما إن RSNA متعلّم من خبراء، النتائج بتاعتنا بتعكس ال model نفسه، مش noise إحنا اللي دخلناه.

A2. من صورة خام لـ model input (ال Preprocessing)

الأشعة الطبية مبتجيش صور عادية، بتيجي على هيئة DICOM files، وده format طبي بيخزّن ال pixels مع بيانات المريض. ال preprocessing بتاعنا بيحوّل كل DICOM لـ PNG نضيفه ال model يقدر يقرأها، بالخطوات دي:

- قراءة ال DICOM بمكتبة pydicom عشان نطّلع مصفوفة ال grayscale pixels الخام.
- **Robust intensity normalization**: سطوع الأشعة بيختلف من جهاز للتاني، فبنقص (clip) كل صورة عند ال 2nd-98th percentiles وبنرجّعها لمدى 0-255. ده بيشيل ال outliers ويخلي الصور قابلة للمقارنة.
- **CLAHE contrast enhancement**: ال (clip) Contrast Limited Adaptive Histogram Equalization (limit = 2.0، tiles = 8x8) بيزوّد ال contrast محلّيًا عشان ال opacities الخفيفة في الرئة تبان. بيعمل equalize في مربعات صغيرة مش على الصورة كلها، وده مثالي للأشعة.
- **Resize** لمقاس ثابت (224x224 للـ classifiers، و640x640 للـ detectors) مع حفظ الأبعاد الأصلية عشان نقدر نرجّع إحداثيات ال boxes صح.

أهم درس اتعلمناه هنا (و غالبًا سؤال امتحان): في الأول كانت النتائج عالية بشكل مريب. السبب كان data leak: ال contrast enhancement كان متطبّق بشكل مختلف على صور الالتهاب وصور الطبيعي، فال model بقى بيعش وبيكتشف ال preprocessing مش المرض. صلّحناها بإننا نطبّق نفس ال CLAHE بالظبط على كل الصور بغض النظر عن ال label. الأرقام الأقل اللي بنعرضها دلوقتي هي الحقيقية. الدرس: ثق في البروتوكول مش في الرقم.

A3. تقسيم ال data صح + معالجة عدم توازن ال classes (imbalance)

قسّمنا ال data لـ train / validation / test على مستوى المريض (patient-level) مش على مستوى الصورة، وب seed ثابت (42) عشان النتائج تتكرر. المريض الواحد ممكن يكون عنده أكثر من صورة، ولو صور نفس المريض اتوزّعت بين ال train وال test، ال model ممكن يحفظ المريض ده ويأخذ درجة أعلى من الحقيقة. التقسيم بالمريض ببيضمن إن ال test set فيه ناس ال model عمره ما

شافها. والتقسيم كمان stratified يعني نسبة الالتهاب للطبيعي محفوظة في كل جزء. الـ test set المعزول = 20% من المرضى: 4,135 صورة طبيعية و1,202 صورة التهاب، ومبلمسهوش خالص أثناء التدريب أو اختيار الـ model.

عدم التوازن (class imbalance): الالتهاب هو الـ class الأقل (minority)، فلو سبنا الـ model زي ما هو ممكن ياخذ accuracy عالية بأنه يقول 'طبيعي' على طول. عالجنا ده بـ class-weighted CrossEntropy: بَدَي وزن أكبر للـ class النادر بنسبة عكسية لتكراره (inverse class frequency)، فالـ model بيتعاقب أكثر لما يفوت حالة التهاب. وكمان بنقيّم بـ AUC و recall والـ confusion matrix مش بالـ accuracy لوحدها. وعلى مستوى الصور بنستخدم augmentation وقت التدريب بس (h-flip، rotation، color jitter) عشان نقّال الـ overfitting.

A4. مهمتين: detection و classification

الـ classification بيجاوب على سؤال 'هل فيه التهاب في الصورة دي؟' باحتمال. والـ detection بيجاوب 'فين بالضبط؟' برسم box أو أكثر، كل واحد معاه نسبة ثقة. عملنا الاتنين لأن الدكتور عايز screening سريع (نعم/لا) وكمان مؤشّر بصري للمكان المشكوك فيه. الـ model اللي بننشره كـ default هو (Faster R-CNN) detector لأن تحديد المكان أنفع إكلينيكيًا.

A5. الـ Models بالتفصيل + عدد الـ parameters

كل الـ models الخمسة بتستخدم transfer learning: بدل ما نتعلّم الرؤية من الصفر، بتبدأ من weights مدربة على ImageNet (dataset ضخم لصور طبيعية) وبعدين بنعمل fine-tune على أشعة الصدر. ده standard في الـ medical imaging لأن الـ datasets الطبية صغيرة، والـ low-level features (الحواف والـ textures) بتنقل كويس.

الفكرة المميّزة	عدد الـ Parameters	النوع	الـ Model
residual connections (شورت كت بيخلى الشبكة العميقة تتدرب)	23.5M	classifier	ResNet50
كل layer متوصّلة بكل اللي بعدها، فالـ features بتتعاد استخدامها (backbone بتاع CheXNet)	7.0M	classifier	DenseNet121
compound scaling - أصغر classifier وأكفأ واحد عندنا	4.0M	classifier	EfficientNet-B0
two-stage: RPN يقترح boxes وبعدين head يصنّف ويظبط - أعلى recall (المنشور)	41.3M	detector	Faster R-CNN
one-stage anchor-free - أسرع وأخف بس أقل recall	3.0M	detector	YOLOv8n

الـ classifiers الثلاثة

- **ResNet50:** شبكة convolutional من 50 layer، فكرتها الأساسية الـ residual connection (شورت كت بيخلى الإشارة تتخطى layers)، وده حلّ مشكلة إن الشبكات العميقة جدًا صعب تتدرب.
- **DenseNet121:** كل layer متوصّلة بكل اللي بعدها فيحصل feature reuse على طول الشبكة. كفاءة في الـ parameters وكانت الـ backbone بتاع CheXNet، الموديل الشهير اللي وصل لمستوى الـ radiologist.
- **EfficientNet-B0:** بيستخدم compound scaling عشان يوازن بين العمق والعرض ودقة الـ input. هو أحسن classifier عندنا وكمان أصغرهم (4M parameters)، فهو الأكفأ.

الـ detectors الاتنين

- **Faster R-CNN (two-stage):** الأول Region Proposal Network يقترح boxes مرشّحة، وبعدين head تاني بيصنّف كل box ويظبط إحداثياته. بيستخدم ResNet50 backbone مع Feature Pyramid Network عشان يشوف الأحجام المختلفة. دقيق خصوصًا في الـ recall، وعشان كده بننشره.
- **YOLOv8 (one-stage):** بيتنبأ بكل الـ boxes في forward pass واحدة بـ head من غير anchors. أسرع وأصغر بكثير، بس في اختباراتنا فوّت حالات التهاب أكثر (أقل recall) من Faster R-CNN.

A6. تدريب الـ classifiers (كل الإعدادات)

السبب	القيمة	الإعداد
fine-tuning خيار افتراضي قوي ومتكيف للـ	Adam	Optimizer
pretrained weights صغير عشان مايعربش الـ	1e-4	Learning rate
بيدي وزن للـ class النادر (الالتهاب) فماتجاهلوش	Cross-entropy موزون بالـ class	Loss
بيقل الـ LR لما الـ validation يبطّل يتحسن	ReduceLROnPlateau (على val AUC، x0.5)	LR scheduler
تدريب أسرع وذاكرة أقل	On على الـ GPU	Mixed precision (AMP)
نوقف قبل الـ overfitting ونرجع أحسن weights	patience = 4 على val AUC	Early stopping
بيعلم الـ model الثبات ويقلّ الـ overfitting	h-flip، rotate 10، color jitter	Augmentation (train بس)
يطابق الـ pretrained backbone	ImageNet mean/std	Normalization
المقاس القياسي للـ backbones دي	x 224 224	Image size

نقطين مهمين. **Class weighting**: الالتهاب هو الأقلية، فيوزن الـ loss بنسبة عكسية لتكرار الـ class، من غيرها الـ model ممكن ياخذ accuracy عالية بأنه يقول 'طبيعي' دائماً. **Best-checkpoint restore**: بنحتفظ بالـ weights من الـ epoch اللي عندها أحسن validation AUC، مش آخر epoch، عشان منشحنش model عمل overfit.

A7. تدريب الـ detectors

TorchVision training loop: optimizer = SGD (momentum 0.9)، learning- على الـ Faster R-CNN بيتدرب على الـ rate schedule = linear warmup بعدها cosine decay، تجميد الـ backbone في أول epoch عشان الـ heads الجديدة تستقر قبل ما نعمل fine-tune للشبكة كلها، early stopping، AMP، على 0.5 mAP، و best-checkpoint restore. بيتحسن على الـ Faster R-CNN multi-task loss (objectness + box regression)، و classification + box-regression للـ head. وقت الـ inference بنطبق Non-Maximum Suppression (NMS) عشان نشيل الـ boxes المكررة المتداخلة (مشروح في القسم B). أما YOLOv8 فيبتدرب من خلال الـ Ultralytics pipeline بالـ augmentation الجاهز بتاعه.

A8. الـ Nature-Inspired Optimization: PSO و GWO و SA

عشان ندور على إعدادات (hyperparameters) أحسن، طبقنا ثلاث خوارزميات مستوحاة من الطبيعة. كلهم طرق للبحث في فضاء من الإعدادات من غير ما نجرّب كل الاحتمالات:

- **Particle Swarm Optimization (PSO)**: سرب من الحلول المرشحة 'بيطير' في فضاء البحث، كل واحد بينجذب لأحسن نتيجة ليه ولأحسن نتيجة للسرب كله، زي العصافير اللي بتلّم على الأكل.
- **Grey Wolf Optimizer (GWO)**: بيقلّد هرم قطيع الدباب؛ أحسن ثلاث حلول (alpha و beta و delta) يقودوا باقي القطيع ناحية المناطق الواعدة.
- **Simulated Annealing (SA)**: مستوحى من تبريد المعدن؛ بيقلّب أحياناً حلول أسوأ في الأول (حرارة عالية) عشان يهرب من الـ local traps، وبعدين بيستقر مع ما بيبرد.

النتيجة الصادقة (سؤال متوقع): في النسخة الأولى قيمنا كل candidate بـ proxy رخيص (epoch واحدة على جزء من الـ data وبـ budget صغير: population=3, iterations=2) عشان البحث يخلص في session واحدة على Kaggle. الـ proxy ده طلع noisy جداً: الإعدادات اللي اختارها مكانتش بتكسب بعد التدريب الكامل، وفي حالات خلّت النتيجة أسوأ من الـ baseline (مثلاً Faster R-CNN mAP@0.5 نزلت من 0.381 لـ 0.175، و EfficientNet AUC من 0.886 لـ 0.874). الدرس إن proxy search خفيف مش بالضرورة بيتفوق على baseline متظبوط، وده finding حقيقي ومحترم مش فشل.

الحل: إعادة بحث أقوى (Stronger re-run)

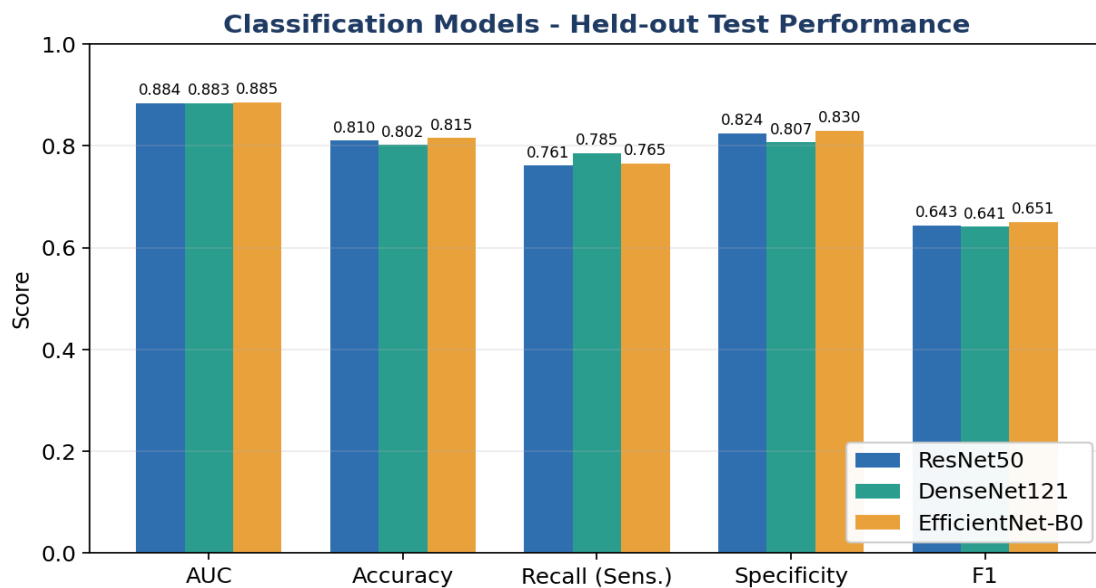
ضفنا في كل notebook قسم 'Stronger nature-inspired search + retrain' بيعيد الـ Phase 4 بـ budget أكبر ودقة proxy أعلى (epochs أكثر و data أكثر لكل candidate، عشان الإعدادات اللي بتتختار تنفع فعلاً في التدريب الكامل)، وبعدين

ببمعمل retrain كامل. القسم ده بيقرأ الـ baseline المحفوظ من results/<model>_report.json عشان يقارن before/after، فمش محتاج يعيد تدريب الـ Phase 3 خالص، وبيستبدل الـ checkpoint المنشور بس لو النسخة الجديدة كسبت الـ baseline. الأرقام النهائية للجزء ده بتتحدث بعد ما نشغله على Kaggle.

A9. مقاييس التقييم (Metrics) بشرح بسيط

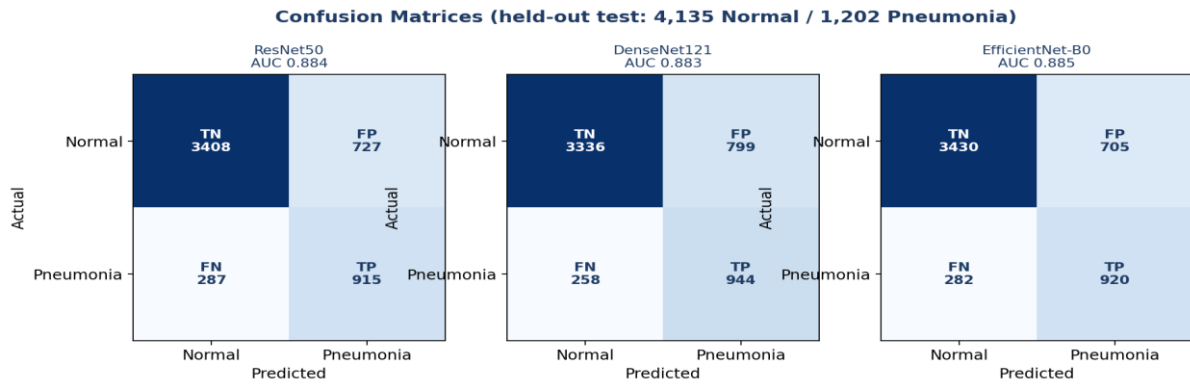
- **Accuracy**: نسبة التنبؤات الصح من الكل. مضللة على الـ data غير المتوازنة، فمابنعتمدش عليها لوحدها.
- **Recall / Sensitivity**: من المرضى الحقيقيين، كام واحد مسكناه. ده أهم مقياس هنا، لأن حالة التهاب فائتة ممكن تكون خطر على الحياة.
- **Specificity**: من الأصحاء الحقيقيين، كام واحد طلعناه سليم صح.
- **Precision**: من اللي قلنا عليهم مرضى، كام واحد فعلاً عنده التهاب.
- **F1**: التوازن (harmonic mean) بين الـ precision والـ recall.
- **AUC**: المساحة تحت الـ ROC curve؛ احتمال إن الـ model يرتب مريض عشوائي أعلى من سليم عشوائي. 0.5 عشوائي و1.0 مثالي. مش معتمد على threshold، فهو المقياس الرئيسي عندنا للـ classification.
- **Confusion matrix**: جدول الـ true/false positives و negatives اللي بنحسب منه كل اللي فوق.
- **mAP@0.5 و IoU**: الـ detection. الـ IoU (Intersection over Union) بيقيس تداخل الـ boxes؛ الـ box بيتحسب صح لو الـ IoU بتاعه مع الحقيقة عدى threshold (مثلاً 0.5). الـ mAP متوسط الـ precision عبر مستويات الـ recall؛ mAP@0.5 بيستخدم threshold = 0.5، و mAP@[.5:.95] بيتوسط على thresholds أصعب.

A10. النتائج، وليه طلعت كده، وإزاي نحسنها



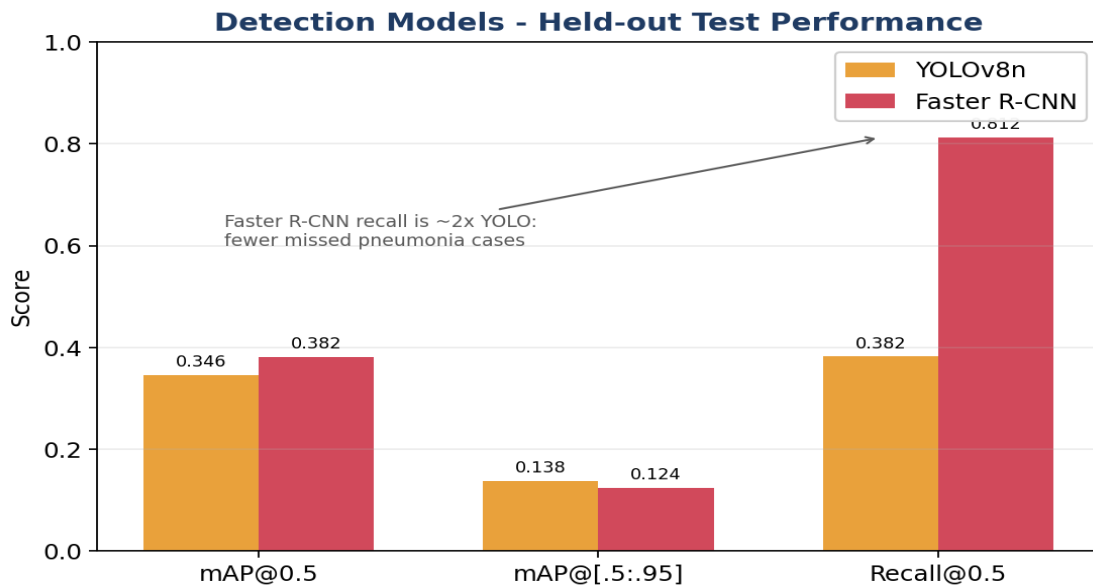
شكل A2. مقارنة أداء الـ classifiers الثلاثة على الـ test set.

Params	F1	Recall	Accuracy	AUC	الـ Model (classifier)
4.0M	0.651	0.765	0.815	0.886	EfficientNet-B0 (الأحسن، منشور)
7.0M	0.641	0.785	0.802	0.883	DenseNet121 (منشور)
23.5M	0.643	0.761	0.810	0.884	ResNet50



شكل A3. الم confusions matrices للـ classifiers الثلاثة على الـ test set.

Params	Recall	mAP@[.5:.95]	mAP@0.5	Model (detector) الـ
41.3M	0.812	0.124	0.381	Faster R-CNN (المنشور)
3.0M	0.382	0.138	0.346	YOLOv8n



شكل A4. مقارنة الـ detectors (YOLO مقابل Faster R-CNN).

ليه الأرقام دي؟ AUC حوالي 0.88 هو اللي بيوصله أي classifier صادق على نوع الـ data ده؛ للمقارنة، CheXNet المشهور سجل 0.768 على dataset ثاني. الـ classifiers بيحافظوا على specificity عالية (حوالي 0.83) ومع كده بيمسكوا حوالي 76-79% من حالات الالتهاب. في الـ detection، الـ detectors بيوصلوا لـ $mAP@0.5$ متقارب (حوالي 0.35-0.38)، وده بالظبط المدى 0.32-0.39 المذكور في الأبحاث)، بس Faster R-CNN بيمسك $recall = 0.812$ من مناطق الالتهاب مقابل 0.382 لـ YOLO.

ليه Faster R-CNN هو الأحسن (والمنشور)؟

الخطأ الأعلى في الـ screening هو إننا نفوت حالة Faster R-CNN (false negative). عنده أعلى recall بفارق كبير (0.812 مقابل 0.382 لـ YOLO)، يعني بيغوت أقل حالات بكثير، وكمان أعلى $mAP@0.5$ بين الـ detectors. عشان كده بننشره كـ default رغم إنه أكبر (41.3M parameters) وأبطأ. الـ trade-off مقصود: في الطب، مسك الحالة أهم من السرعة.

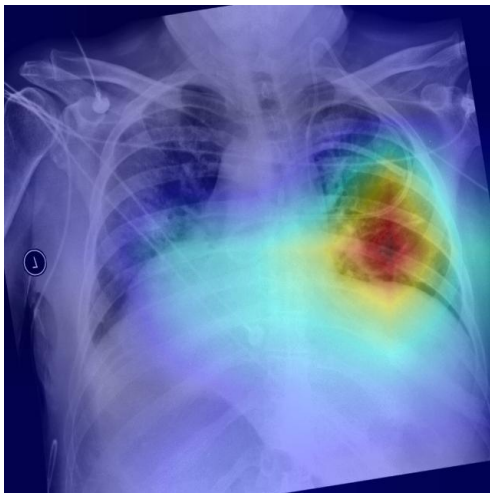
إزاي نطلع نتائج أحسن؟ ندرب على data أكثر وأكثر تنوعاً (مستشفيات وأجهزة مختلفة)؛ نرفع دقة الـ input عشان الـ opacities الخفيفة تفضل بانه؛ نعمل ensemble لأكثر من model؛ نظبط الـ decision threshold عشان نبادل specificity بـ recall حسب احتياج العيادة؛ نضيف augmentation أقوى وواقعي طبيياً؛ نعمل pretrain على corpus كبير لأشعة الصدر قبل الـ fine-tune؛ ولـ detection، ندرب أطول بالـ objective الكامل (مش proxy). وكمان external validation على dataset ثاني هيخلي النتائج أكثر مصداقية.

A11. الـ Explainability (XAI): نوري الدكتور 'ليه'

كل تنبؤ بنعرضه مع heatmap عشان الدكتور يتأكد إن الـ model بصّ على الرئة، مش على نص أو artifact. ده بيبيّن الثقة وبيخلّي القرار قابل للمراجعة. طبّقنا أكثر من طريقة، وكلها بتشغل live مع كل طلب، وكلها best-effort (لو طريقة فشلت بتتخطّى من غير ما تكسر التنبؤ).

لـ classifiers

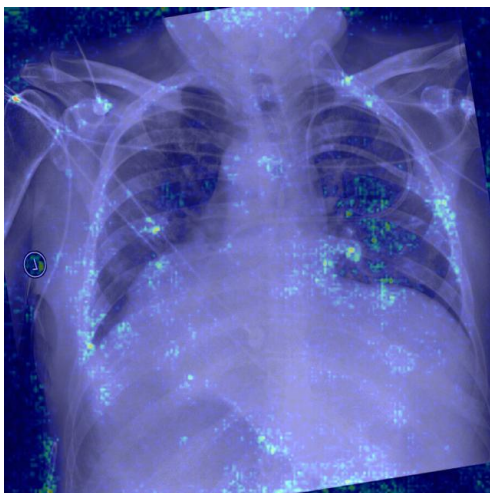
- **Grad-CAM**: بيستخدم الـ gradients عند آخر convolutional layer عشان يطلع heatmap للمناطق اللي زوّدت احتمال الالتهاب. أكثر طريقة شائعة ومباشرة.
 - **Integrated Gradients**: بيجمع الـ gradient على طول مسار من صورة سودا (baseline) للصورة الحقيقية، فيبيّن attribution لكل pixel.
 - **GradientSHAP**: تقدير على طريقة SHAP بياخد متوسط gradient * (input - baseline) على عينات فيها noise.
 - **Score-CAM**: من غير gradients؛ بيقتّع الصورة بكل activation channel ويوزنه بالـ pneumonia score الناتج، فيبيّن map class-discriminative.
- مثال حقيقي - حالة **Pneumonia**: كل الصور دي نواتج فعلية من الـ web application على نفس صورة الأشعة. الـ classifier (EfficientNet-B0) قال **PNEUMONIA** باحتمال **75%**. تحت كل صورة مكتوب التنبؤ، والـ heatmaps بتركّز على الرئة اليسرى (مكان الـ opacity) مش على حرف أو حافة.



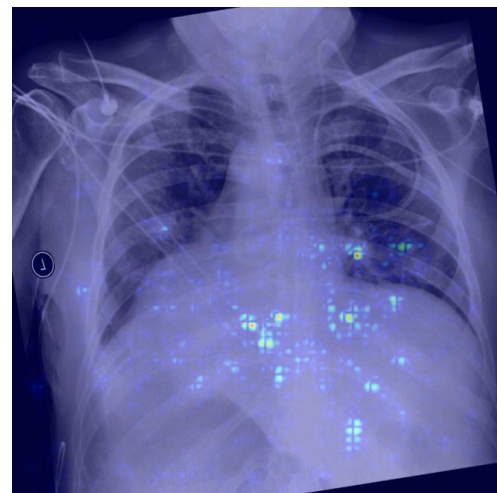
Grad-CAM - بيركّز على الرئة المصابة



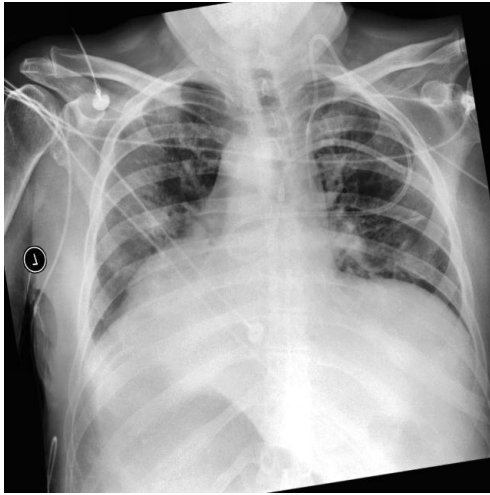
EfficientNet-B0 - التنبؤ: (75%) PNEUMONIA



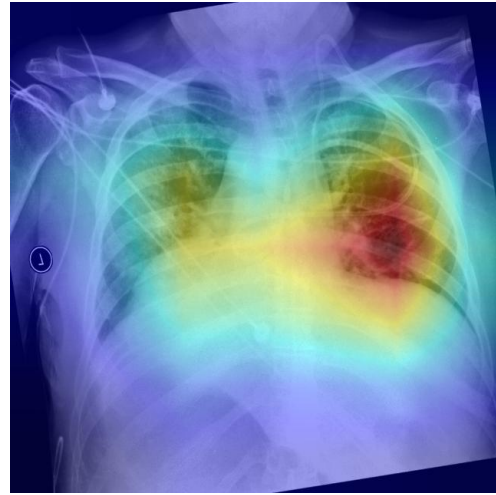
GradientSHAP - PNEUMONIA



Integrated Gradients - PNEUMONIA



(input X-ray) الصورة الأصلية



PNEUMONIA - (من غير gradients) Score-CAM

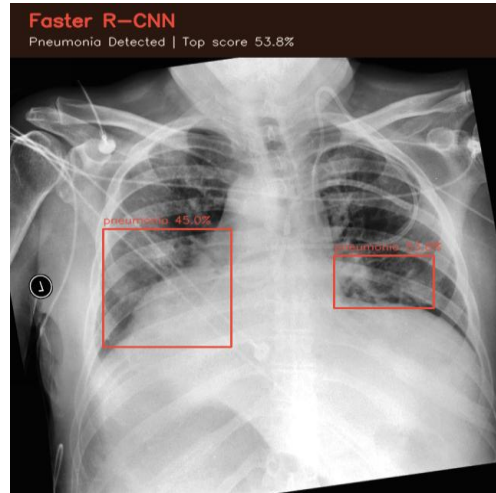
لـ detectors

- **Eigen-CAM:** الـ saliency القياسي بالـ gradients مبيّط بـ سهولة على الـ detectors، فيستخدم Eigen-CAM الـ ليحلّ الـ activations بناءً على الـ backbone (المكوّن الرئيسي - principal component) عشان بيبيّن الشبكة بتركز فين.
- **Occlusion sensitivity:** بنغطي أجزاء من الصورة بمربع ونشوف ثقة الـ detection بتنزل قد إيه؛ كل ما النزول أكبر، كل ما المنطقة دي أهم.

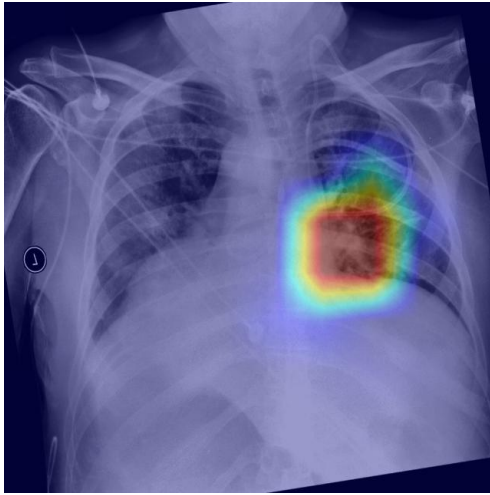
نفس الصورة بالـ **detectors: Faster R-CNN** لقي الالتهاب ورسم box (نسبة 54%، ومكتوب **Pneumonia Detected** فوق الصورة)، لكن YOLO فوّت الحالة (no detection) - وده بالظبط سبب نشرنا لـ Faster R-CNN: الـ recall الأعلى. والـ Eigen-CAM والـ Occlusion بيوضحوا منطقة تركيز الـ detector.



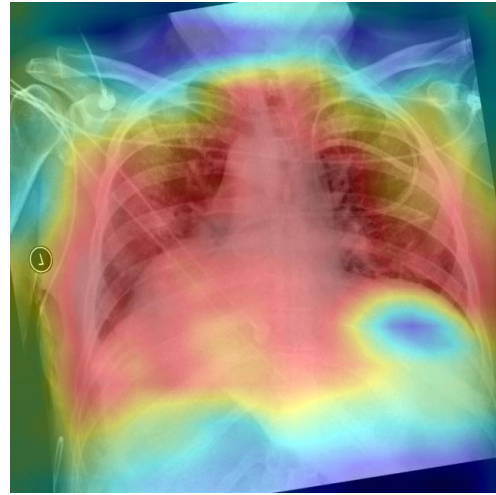
YOLOv8 - فوّت الحالة (no detection)



Faster R-CNN - التنبؤ: (box) Pneumonia Detected



Occlusion sensitivity (Faster R-CNN)



Eigen-CAM (Faster R-CNN backbone)

للمقارنة - حالة Normal: على أشعة صدر سليمة، نفس الـ classifier قال **Normal باحتمال 99%** (احتمال الالتهاب 0.8% بس)، فالنظام يفرّق بوضوح بين الحالتين.



Normal (99%) - التنبؤ: EfficientNet-B0



أشعة صدر سليمة (input)

ملاحظة: كل الصور دي نواتج حقيقية من الـ web application الشغال. الـ heatmaps في حالة الالتهاب بتركز على الرئة المصابة، وده بيبأكد إن الـ models بتتعلم إشارات طبية حقيقية مش artifacts.

A12. القسم A - أسئلة متوقعة من اللجنة

س: ليه نتايجكوا الأولى كانت شبه مثالية وعملتوا إيه؟

ج: كانت متضخمة بسبب data leak في الـ preprocessing: الـ contrast enhancement كان متطبق بشكل مختلف على الـ class اتنين، فالـ model كان بيكتشف الـ preprocessing مش المرض. صلحناها بتطبيق نفس الـ CLAHE على كل الصور وبنعرض الأرقام الصادقة الأقل.

س: ليه التقسيم بالمرضى مش بالصورة؟

ج: لأن المريض الواحد ممكن يكون عنده أكثر من صورة. التقسيم بالصورة ممكن يحط نفس المريض في الـ train والـ test، فالـ model يحفظه ويأخذ درجة أعلى من الحقيقة. التقسيم بالمرضى بيضمن إن الـ test مش متشاف.

س: ليه الـ recall هو أهم مقياس عندكوا؟

ج: لأن في screening الالتهاب، الـ false negative (حالة فايئة) ممكن يكون خطر على الحياة، أما الـ false positive بيأدي لمراجعة ثانية بس. فبنختار ونظبط الـ models عشان نقلل الحالات الفايئة.

س: ليه بتنشروا **Faster R-CNN** مش **YOLO** الأسرع؟

ج: الاتنين بيوصلوا mAP@0.5 متقارب، بس Faster R-CNN بيمسك recall = 0.812 مقابل 0.382 لـ YOLO. بنقبل الـ model الأكبر والأبطأ لأن مسك الحالات أهم من السرعة هنا.

س: عالجتوا عدم التوازن (imbalance) إزاي؟

ج: وزنا الـ loss بنسبة عكسية لتكرار الـ class عشان الـ class النادر مايتجاهلش، وبنقيم بـ AUC و recall و الـ confusion matrix مش بالـ accuracy لوحدها.

س: بتمنعوا الـ overfitting إزاي؟

ج: augmentation، transfer learning، وقت التدريب بس، early stopping، LR scheduler، weight decay على validation AUC، واسترجاع أحسن checkpoint مش آخر epoch.

س: الـ metaheuristic optimization نفع؟

ج: مع الـ proxy الرخيص لأ: PSO و GWO و SA اختاروا إعدادات مكسبتش بعد retrain كامل. عشان كده اخترنا أحسن validation checkpoint، وضمنا قسم re-run أقوى يعيد البحث ببدجت ودقة أعلى، وبيستبدل الـ model بس لو كسب الـ baseline.

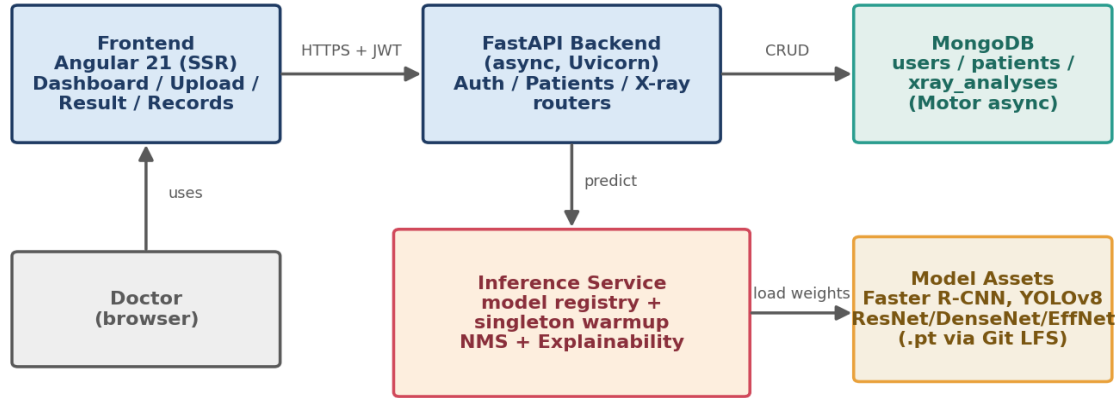
س: الـ model آمن إكلينيكيًا؟

ج: هو decision support مش تشخيص أوتوماتيك. الدكتور بيأكد كل نتيجة، وكل تنبؤ بييجي بنسبة ثقة و heatmap عشان يتراجع. محتاج external validation وموافقات تنظيمية قبل الاستخدام الإكلينيكي الفعلي.

القسم B - ال Web Application Pipeline

التطبيق من ثلاث طبقات: frontend الدكتور بيشوفه، backend يعمل الشغل، و database بيخزن كل حاجة. وجوه ال backend في inference service متخصص بيشغل ال AI models. القسم ده بيشرح كل جزء وبعدين بيتتبع طلب واحد من الضغطة للنتيجة.

System Architecture

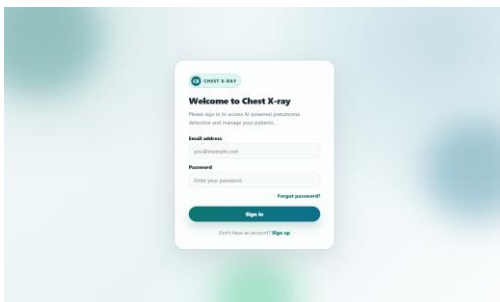


شكل B1. ال three-tier architecture مع ال inference service.

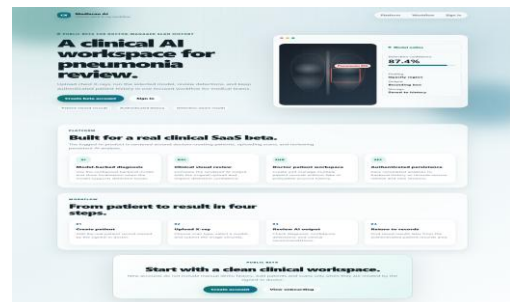
B1. ال Frontend (Angular 21)

ال frontend هو (SPA) single-page application متعمول بـ Angular ومكتوب بـ TypeScript. ال single-page معناها إن المتصفح بيحمل تطبيق واحد وبعدين بيبدل الشاشات من غير reload كامل، فيحس بالسرعة. مكوّناته:

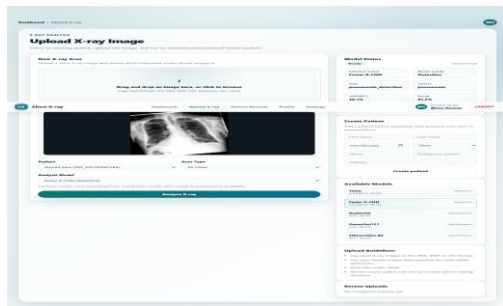
- **Components / pages:** login, signup, dashboard, upload, processing, result, patient records, profile, settings. كل واحدة قطعة UI مستقلة.
- **Routing + guards:** ال router بيربط ال URLs بالصفحات؛ وال route guards بيمنعوا الصفحات المحمية (زي ال dashboard) إلا لو المستخدم عامل login، وبيرجعوا اللي عامل login بعيد عن صفحة ال login.
- **State service:** analysis-state service مركزي بيمسك التحليل الحالي وال history عشان الصفحات تشارك ال data من غير ما نمررها بإيدينا.
- **API service:** service واحد بيجتمع كل النداءات للـ backend، وبيحط ال login token مع كل request، فباقي التطبيق مبيتعاملش مع ال HTTP مباشرة.
- **Server-side rendering (SSR):** الصفحات العامة بتترسم مبدئيًا على Express server صغير عشان أول ظهور يبقى سريع، وبعدين التطبيق بيعمل hydrate ويبقى SPA تفاعلي.



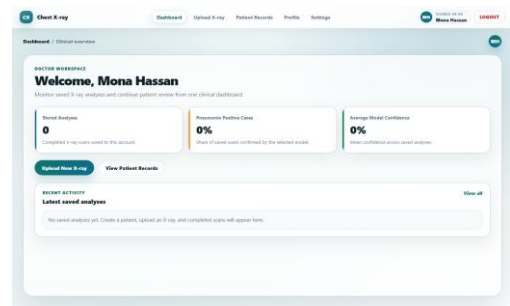
صفحة ال login



صفحة الترحيب (welcome)



صفحة رفع الأشعة (upload) مع اختيار الـ model



الـ dashboard

B2. الـ Backend (FastAPI)

الـ backend عبارة عن FastAPI service بالـ Python. FastAPI غير متزامن (asynchronous)، يعني ببقدر يخدم طلبات كثير في نفس الوقت من غير ما يتعطل: وهو مستني الـ database أو الـ model، بيخدم غيره. مكوّناته:

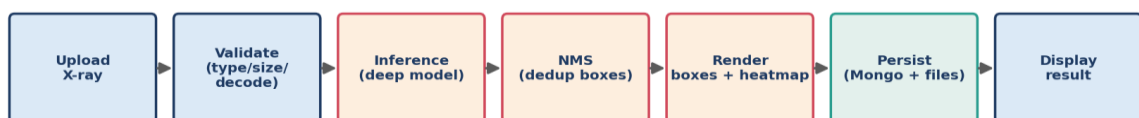
- **Routers:** الـ API متقسّم حسب المورد لـ `patients` و `x-ray`، وده بينظّم الكود.
- **Pydantic models:** كل `request` و `response` بيتحقّق منه ضد `schema` بأنواع محددة، فالبينات الغلط بترفض أو توماتيك برسالة واضحة.
- **Lifespan startup:** لما الـ server يقوم بيتوصّل بـ MongoDB ويعمل `warmup` للـ `default model` (يحمّله في الذاكرة مرة واحدة)، وبينصّف عند الإغلاق.
- **Model warmup (singleton):** الـ model بيتحمّل مرة واحدة عند الإقلاع وبيتعاد استخدامه لكل طلب، فالمستخدم مبيدفعش تكلفة التحميل (ثواني) كل مرة. ده أهم قرار للاداء.

B3. الـ Inference Service خطوة بخطوة

ده قلب الـ backend. لما الصورة توصل، بيعمل:

- **Validate:** نوع الملف مسموح (`.../jpg/png`)، الحجم أقل من 10MB، وإنه فعلاً بينفك كصورة.
 - **Model registry lookup:** بيدور على الـ model في `registry` (جدول بيربط مفتاح الـ model بالـ `weights` والـ `task` والـ `thresholds`). الـ `registry` فيه دلوقتي 5 `models`: YOLO، Faster R-CNN (detection)، ResNet50، DenseNet121، EfficientNet-B0 (classification).
 - **Predict:** بيشغل الـ model عشان يطّلع التنبؤات الخام (`boxes + scores`، أو احتمال `class`).
 - **Non-Maximum Suppression (NMS):** الـ detectors بيطلعوا كذا `box` متداخل لنفس المنطقة؛ الـ NMS بيسبب أعلى `box` ثقة ويشيل اللي بيتداخل معاه أكثر من 30%، فالدكتور يشوف `box` واحد نضيف لكل منطقة. (Faster R-CNN بيعمل NMS داخلياً، فينبطّقه على YOLO بس).
 - **Render + XAI:** بيرسم الصورة المعلّمة (annotated) وheatmap الـ `explainability`.
 - **Respond:** بيرجع نتيجة منظّمة: القرار، الـ `boxes`، الثقة، وقت المعالجة، ومسارات الصور، وبعدين الـ backend بيحفظها.
- الـ **confidence thresholds:** الـ detection فوق 0.10 بيتعرض كـ 'مشتبه' (suspected)؛ وفوق 0.25 بيتعامل كـ `finding` مؤكّد. الـ `thresholds` دي قابلة للضبط لكل model في الـ `registry`.

Prediction Data Flow

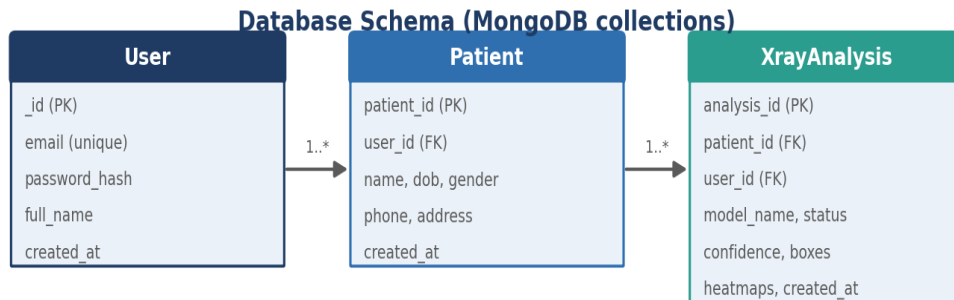


B4. الـ Security

- **JWT authentication**: عند الـ login السيرفر يبصدر JSON Web Token موقع (تذكرة مضادة للتزوير وبنتهي). المتصفح يبيعه مع كل request، والسيرفر يتأكد من التوقيع عشان يعرف إنت مين من غير sessions على السيرفر.
- **bcrypt password hashing**: الباسوردات مبتخزنش plaintext أبداً. bcrypt بيضيف salt عشوائي وبطيء بقصد، فده بيصعب كسر قواعد بيانات الباسوردات المسروقة.
- **Per-user authorization**: كل database query متفلتر بـ id الدكتور المسجل، فمفيش دكتور يقدر يقرأ مرضى دكتور تاني. ده مختبر.
- **Input validation + CORS**: الـ uploads بتنقح قبل ما توصل لأي model، والـ API بيقبل طلبات من origins معروفة بس.
- **Production secret guard**: السيرفر بيرفض يقوم في الـ production بالـ secret key الافتراضي، فبيمنع غلطة نشر شائعة وخطيرة.

B5. الـ Database (MongoDB)

بنستخدم MongoDB، وهو document database، من خلال الـ Motor driver غير المترامن. فيه ثلاث collections: users و patients و xray_analyses. اخترنا document database لأن التحليل الواحد طبيعي إنه يبقى document واحد مكتفي بذاته (الـ boxes والـ scores ومسارات الـ heatmaps والـ timestamps بنقراهم مع بعض دايمًا)، وده بيتطابق مع document شبه JSON ويتجنب الـ joins المعقدة. وفي unique indexes بتمنع تكرار الإيميلات والمعرفات، وcompound index على المالك + وقت الإنشاء بيخلي تحميل history الدكتور سريع.



شكل B3. الـ Entity Relationship Diagram للـ collections.

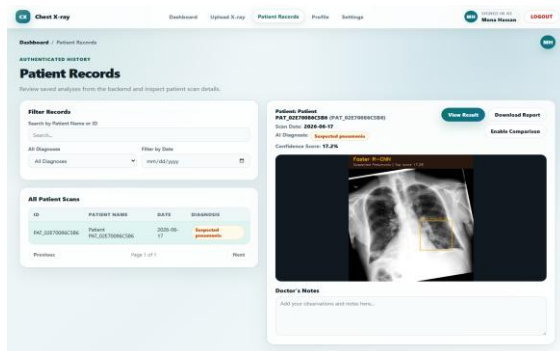
B6. الـ API

بيعمل إيه	Method	Endpoint
تسجيل، login (بيرجع token)، بيانات المستخدم الحالي	POST/GET	api/auth/signup, /login, /me/
إنشاء/قراءة/تعديل/حذف المرضى (لكل دكتور)	CRUD	api/patients/
رفع صورة، تشغيل inference، حفظ النتيجة	POST	api/xray/upload/
عرض أو إدارة التحاليل المحفوظة	GET/DELETE	api/xray, /api/xray/{id}/

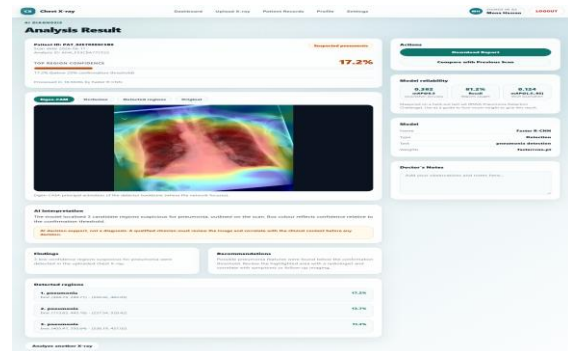
Method	Endpoint	بيعمل إيه
GET	health, /docs/	فحص الصحة؛ توثيق Swagger تفاعلي

B7. رحلة طلب كاملة من الأول للآخر

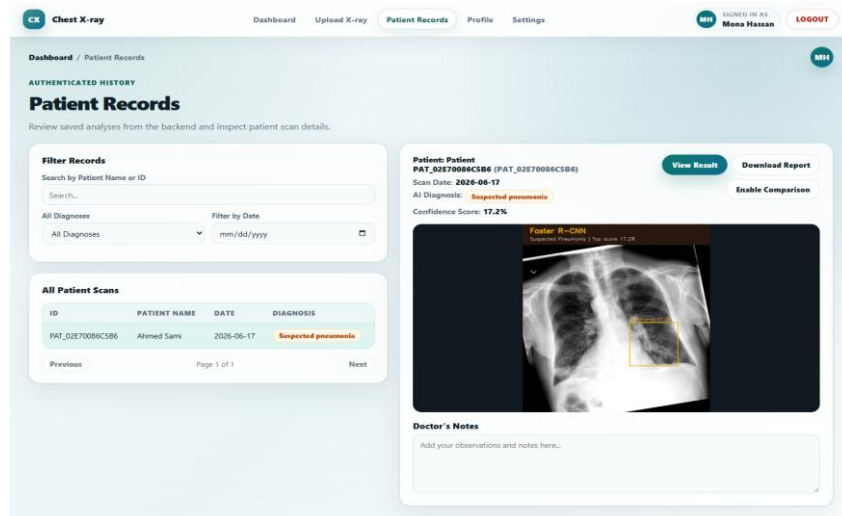
(1) الدكتور بيعمل login؛ ال frontend بيخزن ال JWT. (2) بيفتح مريض ويرفع أشعة؛ صفحة الرفع بتوريه preview ويختار ال (3. model) ال API service ببيعت الصورة (مع ال token) لـ FastAPI (4) api/xray/upload. token، بيفحص الملف، ويسلمه لـ (5. inference service) ال model المحمل بيتنبا؛ ال NMS بينصف ال boxes؛ heatmap بيترسم. (6) النتيجة بتتكتب في collection ال xray_analyses والصورة المعلمة بتتخفظ. (7) ال backend بيرجع النتيجة المنظّمة؛ وال frontend بيعرضها في شاشة النتيجة ويضيفها لـ history المريض.



سجل التحاليل (records) لكل مريض



شاشة النتيجة: القرار + box + نسبة الثقة + التوصيات



شكل B4. تفاصيل تحليل محفوظ (record detail) بكل النواتج.

B8. ال Deployment والإعداد

- الإعدادات بتعيش في environment variables (ال database URL، ال secret key، ال allowed origins، مدة ال token)، وبعيدة عن ال version control.
- ال Model weights بتتخزن بـ Git LFS لأنها ملفات binary كبيرة، فال repository يفضل صغير وال clone بيحبها عند الحاجة.
- ال Scaling: API مبيمسكش حالة لكل مستخدم في الذاكرة (كل الحالة في ال database وال token)، فينفع ينتسخ ورا load balancer؛ وال model ممكن ينتقل لـ service لوحده للأحمال الثقيلة.
- التشغيل: شغل MongoDB، شغل ال backend بـ Uvicorn، ابني وقدم ال Angular frontend؛ الخطوات الكاملة في ال README ودليل المستخدم في ال thesis.

B9. القسم B - أسئلة متوقعة من اللجنة

س: ليه FastAPI و async؟

ج: ال inference وال database calls بتنضمّن انتظار؛ السيرفر ال async بيفضل يخدم باقي المستخدمين أثناء الانتظار، فالتطبيق بيفضل سريع تحت الضغط من غير thread لكل request.

س: ليه MongoDB مش SQL؟

ج: التحليل الواحد document متداخل مكثفي بذاته بنقراه كوحدة، وده بيناسب ال document store ويتجنّب ال joins. وبرضه بنفرض البنية بـ indexes و validation.

س: ال login بيشتغل إزاي بالضبط؟

ج: عند ال login بتأكد من الباسورد المعمول له bcrypt-hash وينصدر JWT موقع بمدة صلاحية. المتصفح بيعت ال token مع كل request، والسيرفر بيتأكد من التوقيع عشان يعرف المستخدم من غير sessions.

س: بتمنعوا دكتور يشوف مرضى دكتور تاني إزاي؟

ج: كل query متفلتر بـ id الدكتور المسجل، فالسجلات اللي مش بتاعته بترجع not-found. وعندنا automated test بيثبت إن الوصول عبر المستخدمين مرفوض.

س: ليه بتحملوا ال model مرة واحدة عند الإقلاع؟

ج: تحميل ال model بياخد ثواني؛ لو عملناه لكل request هيجلي التطبيق بطيء. بنحمله مرة (warmup) ونعيد استخدامه، فكل تنبؤ سريع والذاكرة متوقعة.

س: ال NMS ده إيه وليه محتاجينه؟

ج: ال Non-Maximum Suppression بيثيل ال boxes المكررة المتداخلة، ويسبب أعلى واحد ثقة لكل منطقة (هنا لما التداخل يعدي 30%)، فالدكتور يشوف box واحد نضيف بدل كذا واحد.

س: بتأمنوا ال uploads إزاي؟

ج: بتأكد من النوع والحجم وإن الملف بيتفك كصورة قبل ما يوصل ال model؛ وال API و authentication و CORS؛ وال secrets في environment variables مش في الكود.

س: هتعملوا scale للمستشفى إزاي؟

ج: ال API stateless، فينشغل كذا نسخة ورا load balancer، وننقل ال model لـ inference service أو GPU server مخصص، ونستخدم managed MongoDB cluster. ال containerization وال monitoring هما أول خطوة جاية.